



Atlas

DCT Infrastructure Intelligence

An internal tool for querying data center cutsheets
using natural language

The problem

19+ MB cutsheets

Each Ellendale master cutsheet has thousands of rows across multiple data halls. Finding a single optic count means scrolling through the entire workbook.

Manual cross-referencing

Answering "how many QSFP-DD in DH202?" requires filtering by data hall, section, optic type, and status. One mistake skews the count.

No single source of truth

NetBox, cutsheets, and build sheets each hold partial views. Comparing them takes hours of manual reconciliation.

Tribal knowledge

Only a few people know how to navigate these files. When they're unavailable, the team waits.

What Atlas does

Upload a cutsheet. Ask a question. Get an answer.

Q&A

Natural Language Q&A

Ask questions like "How many QSFP-DD optics are in DH202?" and get accurate, sourced answers in seconds.

- 30+ query types
- Parameterized SQL
- No hallucinated data

LIVE

Live NetBox Dashboard

Real-time fleet visibility across all sites. Device counts, optic breakdowns, and per-rack density updated every 15 minutes.

- Auto site discovery
- Parallel ingestion
- Chart.js visualization

RACK

Rack Analyzer

Drill into any rack: see every device, optic, and cable. Download labels as CSV for install day.

- Internal + outgoing cables
- Cab type detection
- Bulk label export

How it works



Architecture

- **Zero LLM-generated SQL** — Every query is a pre-built parameterized template. The AI classifies the question, Postgres returns the data. No prompt injection. No hallucinated tables.
- **Breakout-aware optic counting** — QSFP-DD breakouts generate 4 connection rows per physical optic. Atlas deduplicates by port identity so counts match physical reality.
- **Background ingest** — Large cutsheets (19MB+) load into Postgres in a background thread. The UI returns optic counts immediately while the database catches up.

30+ query types

Every question is classified and routed to the right SQL template

Optics

- Optic counts by type
- Optic counts by status
- Per-side breakout dedup
- Port utilization

Devices

- Model search
- Device list by hall
- Device detail
- Node compute info

Connections

- Connection status
- Cable status
- Cable type summary
- Link status

Topology

- Rack summary
- Section summary
- Section completion %
- Data hall summary

Network

- LLDP failures
- Neighbor mismatch
- IP lookup
- Role lookup

Analytics

- Cross-site comparison
- Upload diff
- Status trends
- Section trends

Sample queries

"How many QSFP-DD optics are in DH202?"

optic_count

Counts physical optics with breakout dedup, filtered by data hall

"What devices are in rack 041?"

rack_summary

Every device, optic, and cable touching that rack

"Summarize the SPINE section"

section_summary

Device and connection counts within a logical cutsheet section

"What connections are still pending?"

connection_status

Filters by normalized status across all data halls

"What cable types are used in DH204?"

cable_type_summary

DAC vs AOC vs fiber breakdown per data hall

"Compare this upload to the previous one"

upload_diff

Shows added, removed, and changed connections between versions

By the numbers

30+

Query Types

19 MB

Cutsheet Parsed

<3s

Avg Response

4

Data Halls

0

SQL Injection Risk

What this means for DC Ops

Faster answers — Questions that took 20-30 minutes of spreadsheet work now take seconds. Upload once, query all day.

Fewer errors — Parameterized queries eliminate manual filter mistakes. Breakout dedup catches the 4x overcounting problem.

Operational visibility — The live dashboard gives leadership a real-time view of fleet status without asking anyone to pull a report.

Knowledge democratization — Anyone with browser access can query cutsheet data. No spreadsheet expertise required.

Tech stack

Backend

Python / Flask

REST API with rate limiting, HMAC auth, SSE streaming

Database

PostgreSQL

Parameterized SQL, materialized views, CTE-based dedup

AI

Anthropic Claude

Intent classification + answer summarization (no SQL generation)

Ingestion

Calamine + openpyxl

High-speed Excel parsing with profile-based column mapping

Infrastructure

Docker / Kind / Helm

Containerized, ready for cluster deployment

Frontend

Vanilla JS + Chart.js

No framework overhead. Dashboard uses Tailwind CDN



Atlas

DCT Infrastructure Intelligence

Built by Lamar Wells · Inventory Control · CoreWeave